

ROI-Based Object Detection & Segmentation Pipeline

Project Documentation

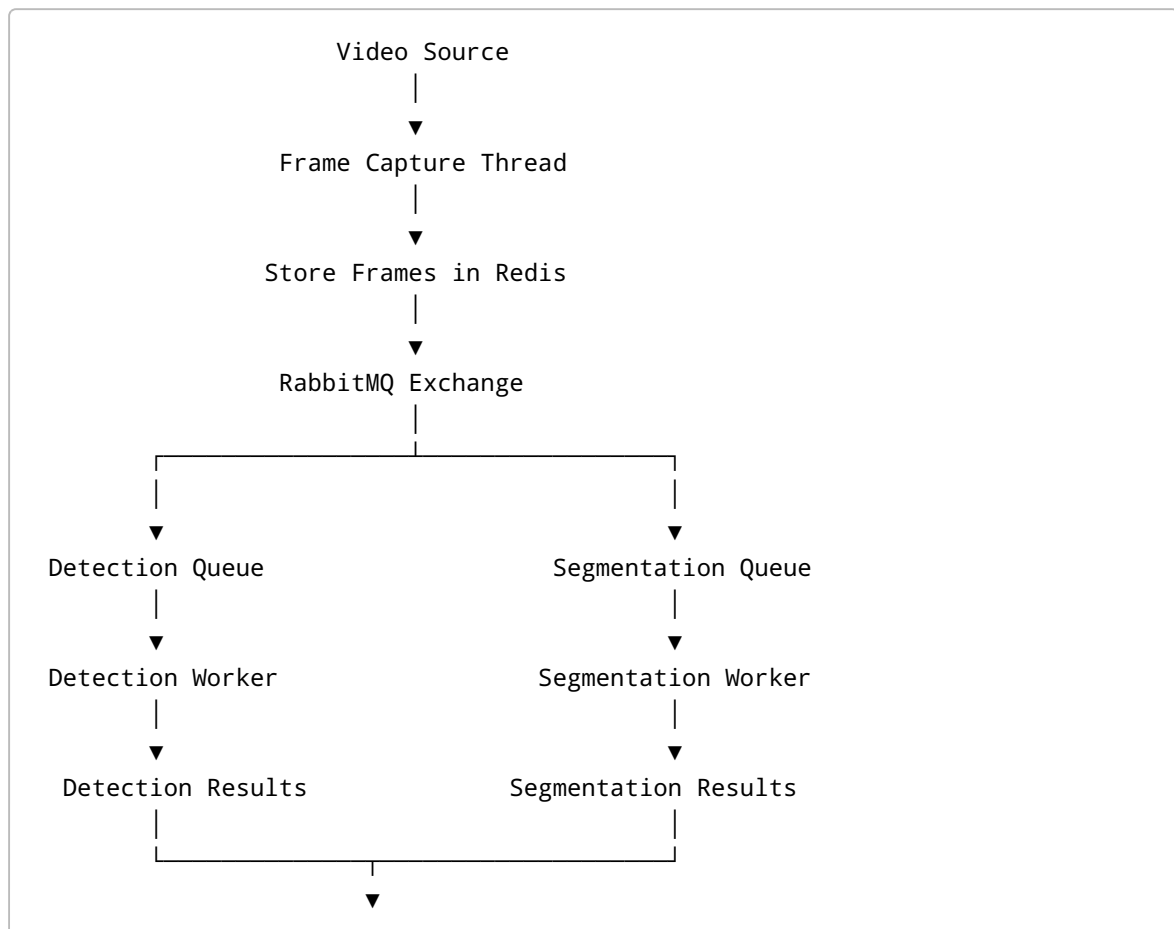
Overview

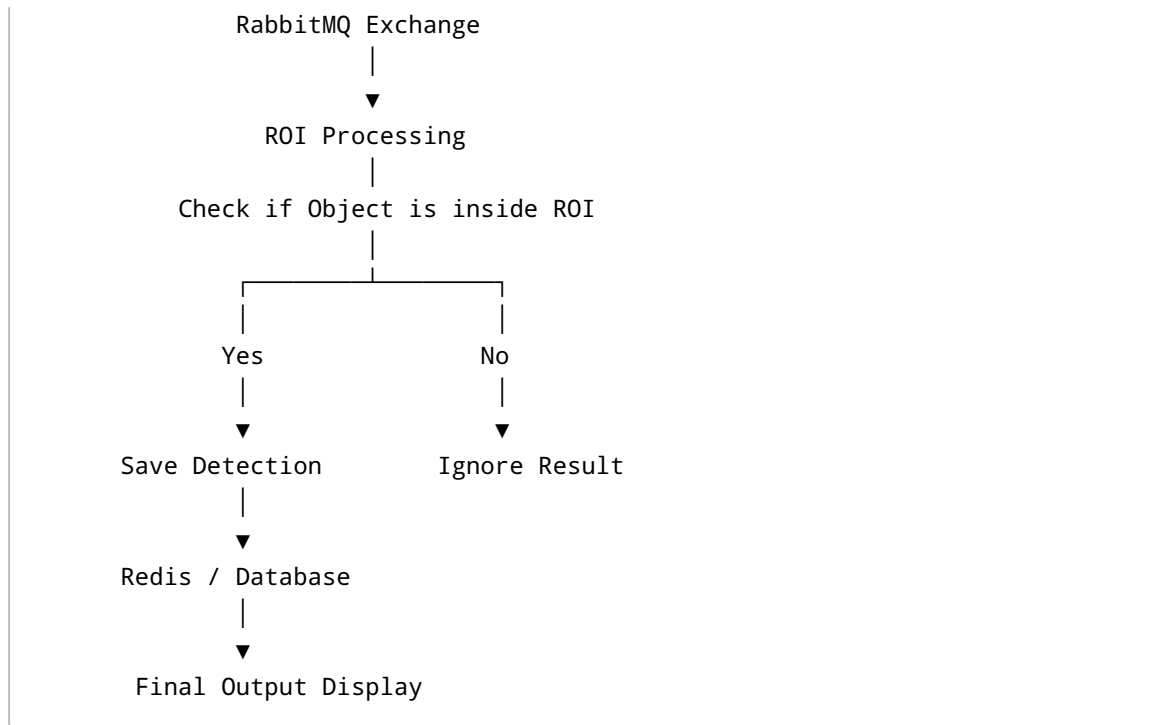
This project is a real-time **ROI-Based Object Detection and Segmentation System** developed using **YOLOv7**, **OpenCV**, **RabbitMQ**, and **Redis**. The system processes live video streams efficiently using asynchronous communication, multithreading, and message queues.

Instead of processing every frame sequentially, the system distributes workloads across different processors using RabbitMQ while Redis acts as a temporary storage layer for image frames.

The complete pipeline ensures high throughput, low latency, and scalable processing.

System Architecture





Technologies Used

- Python
- OpenCV
- YOLOv7
- PyTorch
- Redis
- RabbitMQ
- NumPy
- Threading
- Multiprocessing

Complete Workflow

Step 1 – Video Frame Capture

A dedicated frame capture thread continuously reads frames from the camera or video stream.

Instead of sending image data directly between modules, every frame is encoded and stored in Redis.

Redis acts as a high-speed temporary memory buffer.

Example Redis Lists:

```
vehicle_frames
safety_frames
```

Each frame receives a unique Frame ID.

Step 2 – Redis Frame Storage

Every captured frame is pushed into Redis.

Example:

```
Frame 101
Frame 102
Frame 103
...
```

Redis provides:

- Fast read/write operations
 - Shared access across multiple workers
 - Reduced memory duplication
 - Easy synchronization
-

Step 3 – RabbitMQ Exchange

Once a frame is stored in Redis, only its **Frame ID** is published to a RabbitMQ Exchange.

RabbitMQ does **not** transfer image data.

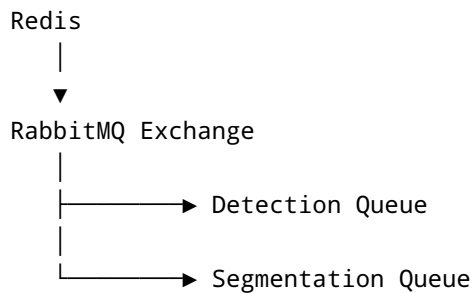
Instead, it transfers lightweight metadata.

Example Message:

```
{
  "frame_id": 102,
  "camera_id": 1,
  "timestamp": "2026-06-26 12:10:32"
}
```

This significantly reduces network overhead.

RabbitMQ Processing Pipeline



The exchange distributes the frame IDs to different processing queues.

Detection Processor

The Detection Worker receives the Frame ID from RabbitMQ.

Steps:

1. Read Frame ID
2. Fetch Frame from Redis
3. Load YOLOv7 Detection Model
4. Perform Object Detection
5. Generate Bounding Boxes
6. Generate Detection Metadata

Example Output:

```
Person
Car
Bike
Helmet
Fire
Smoke
```

The detection results are then published back to RabbitMQ for further processing.

Segmentation Processor

The Segmentation Worker follows a similar process:

1. Receive Frame ID
2. Fetch Frame from Redis

3. Load Segmentation Model
 4. Generate Pixel Masks
 5. Publish Segmentation Results
-

Why RabbitMQ is Used

RabbitMQ enables asynchronous communication between components.

Benefits include:

- Loose coupling between modules
- Parallel processing
- Better scalability
- Fault tolerance
- Load balancing
- High throughput

Each worker processes frames independently without blocking other modules.

Why Redis is Used

Redis acts as a temporary frame cache.

Advantages:

- Very fast in-memory storage
- Shared access across workers
- Eliminates repeated frame transfers
- Supports multiple consumers
- Reduces communication overhead

Only Frame IDs are exchanged through RabbitMQ, while image data remains in Redis.

ROI Processing

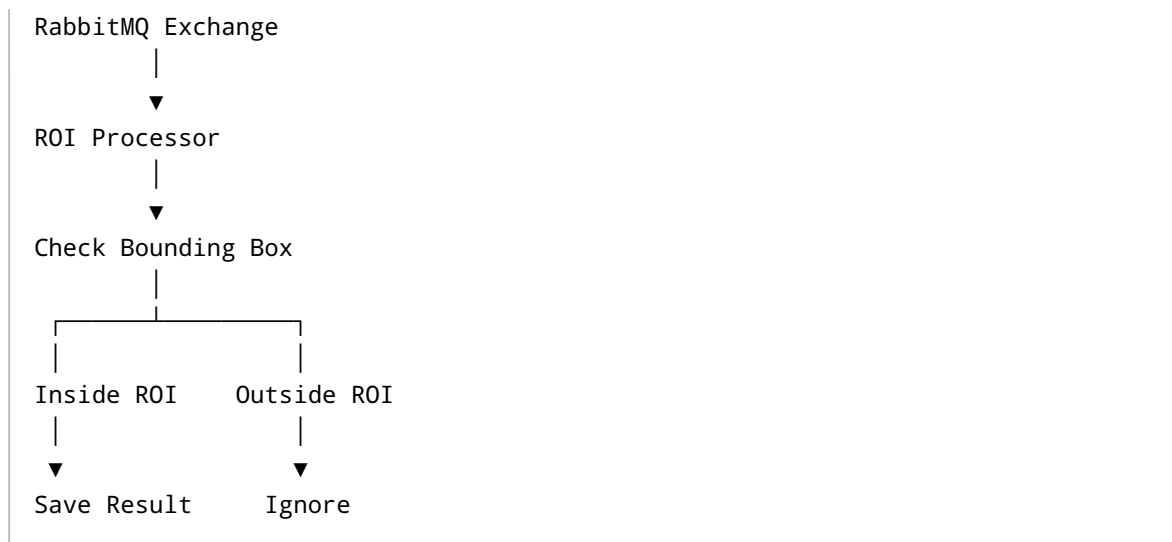
After detection results are generated, they are again published to RabbitMQ.

The ROI Processor subscribes to these detection results.

Workflow:

Detection Result





Only objects whose bounding box lies within the predefined Region of Interest are considered valid detections.

Objects detected outside the ROI are discarded.

Object Saving Logic

Only valid ROI detections are stored.

Saved information includes:

- Frame ID
- Object Class
- Confidence Score
- Bounding Box Coordinates
- Timestamp
- Camera ID

This ensures that only relevant events are retained.

Threading Architecture

The project uses multiple threads to improve performance.

Threads include:

- Frame Capture Thread
- Redis Storage Thread
- Detection Worker Thread
- Segmentation Worker Thread
- ROI Processing Thread

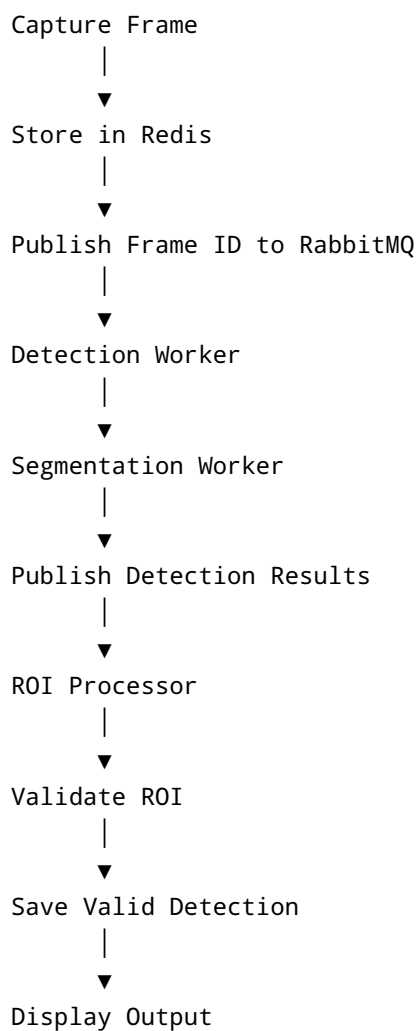
- Output Display Thread

Advantages:

- Non-blocking execution
- Improved FPS
- Better CPU utilization
- Parallel processing

Processing Logic

The system follows this sequence:



Advantages

- High-speed frame processing

- Parallel execution
 - Modular architecture
 - Scalable worker design
 - Reduced latency
 - Efficient memory usage
 - Easy deployment
 - Real-time monitoring
-

Future Enhancements

- Multi-camera support
 - Dynamic ROI selection
 - Object tracking (ByteTrack / DeepSORT)
 - Event notifications
 - Database integration
 - Cloud deployment
 - Web dashboard
 - Analytics and reporting
-

Conclusion

This project demonstrates a scalable real-time computer vision pipeline by combining **YOLOv7**, **RabbitMQ**, **Redis**, and **OpenCV**. Frames are stored efficiently in Redis, lightweight metadata is exchanged through RabbitMQ, detection and segmentation run in parallel, and an ROI processor filters detections before saving only valid events. This architecture minimizes communication overhead, improves processing speed, and provides a reliable foundation for industrial safety, surveillance, and smart monitoring applications.